

Project: Auto Grading System

Expected Effort (approximate): 20 hours per person
Due Dates: Week 13 – 11 Nov 11:59 PM
Weightage: 30% of final grade

(Tentative) Grading Components	Marks
Code - The code is modularized with multiple classes and methods. - The code is clean. - The code is designed and layered to known design principles. For example, Single Responsibility Principle (SRP). - The configuration parameters are externalized. https://reflectoring.io/externalize-configuration - Adherence to Java coding convention. http://www.oracle.com/technetwork/java/codeconventions-150003.pdf - The code is appropriately documented. - The variable, class, and method names are meaningful. - The code is indented using tabs or spaces consistently. - The classes are logically organized with packages.	7
Application - Usability of your application, core features complete, tested end-to-end. - It can be a console, GUI or web application. <i>The main consideration is usability not aesthetics (i.e. have a pretty UI).</i> - Nielsen's 10 heuristic evaluation https://www.nngroup.com/articles/ten-usability-heuristics/ - Submission: source code + README file + presentation slides	16
Presentation - Time allocation: 10-15 minutes for presentation & demonstration. 5 minutes for Q & A. - Presentation Content: - State your team's object-oriented design (system architecture). Use UML class diagrams if necessary. - List all the open-source libraries used. If you evaluated a few, state why you picked that particular library. - List the algorithms explored and explain which one is effective and why. What are the advantages and disadvantages of different algorithms. - Demonstrate the features of your application. - Presentation is well-rehearsed. - Slides are clean and visible. - There is no unnecessary switching of laptops/applications. - The transition is smooth.	7
Penalty - Up to 10% penalty of your total marks if the application has any usability issues (include input validations) - Up to 50% penalty of your total marks if any of your Java source files does not compile or run.	

Distribution of Work

- Each member of the team is expected to be familiar with all aspects of the deliverables submitted.
- Your team may be selected for an interview at the end of the term to walkthrough your team's deliverables with us. **This is to ensure that every member of your team was involved in the project.**

Peer Evaluation

- If you have any teaming issues, do highlight to your instructors early and not wait until the peer evaluation.
- As a team, you will be given a single grade for the project. However, your individual grade for the project can be higher or much lower than the team grade based on peers' evaluation (and instructors' evaluation based on the team's interview and observation across the semester).
- We will penalize freeloaders, as well as team members who decide to complete the project alone.
- Guidelines
 - Please be frank when evaluating your peers and do the evaluation seriously.
 - You **must use the comments** field to explain why you think a particular individual deserves the score. Note that the system will not force you to add comments, but we require it. You need to input comments about every member in your team for every metric. Be frank and honest with your comments.
 - You can send your comments to prof directly by email (with proofs), if, in your group, you feel that a certain member has contributed significantly less or not at all, and you believe they do not deserve the same score as the rest of the team. If you think your team members contribute equally and all team members should obtain the same project score, you can just skip the peer evaluation.
 - Your team peer evaluation score will not be compared against other teams' scores. Having a lower team score than another team will not affect your final grade.

Submission

- All submissions are to be done via **eLearn > Assignments**. All submissions via email will be ignored.
- The zip file should contain the application source code, a README file and presentation slides.
- The application source code should include following folders and files:
 - **compile.sh** and **run.sh** batch files
 - The compile.sh file compiles all your Java source files and automatically stores them in the classes directory.
 - The run.sh runs your application.

Note: You can assume that the instructors' laptops have the PATH environment variable set correctly so that javac.exe and java.exe can be executed at the DOS command prompt from any directory. The instructors will run compile.sh and run.sh on their laptops during assessment.

- **src**
This directory contains all your Java source files.

- **classes**
This directory should be left empty during submission. After compile.bat runs, the class files will be stored here automatically.
- **media files**
This directory contains any image and audio files used by your application. It will be empty if you build a console application.
- **lib (if using any external libraries)**
This directory contains any jar files that you use for your application.
- Alternatively, you can use [Maven](#). If your team chooses to use Maven, you may follow the standard Maven directory structure (src/main/java) instead of the above batch files and folder structure, provided the project compiles and runs successfully.
- A *GX-TY.pptx (or pdf)* of your presentation slides.
 - Include an "AI Use" section listing the AI tools and prompts used for major architectural decisions.
 - If your submission exceeds the eLearn file size limit, you may add an external link to your presentation deck as an appendix.
 - You can hide your appendix slides.
- Only the use of external free or open source libraries is allowed. No other form of code-sharing is allowed. Co-development of code with members from other teams is a definite NO-NO.

Use of AI tools

The use of AI tools (e.g., GitHub Copilot, ChatGPT) is permitted for this project, provided the team maintains full accountability for the submitted work.

- **Every team member must be able to explain the logic and implementation of any AI-assisted code during the presentation.** AI outputs can be inaccurate or outdated; you must manually verify that all code compiles and adheres to the project rules. "AI-generated" is not a valid justification for a logic error or a "hacky" solution.
- You should use these AI tools as an architect rather than a shortcut.
- All AI usage must be transparently documented:
 - Use comments to attribute specific methods. For example:
 - `// Generated by ChatGPT-4; modified for tie-breaker logic`
- Include an "AI Use" section in your presentation deck's appendix listing the tools and prompts used for major architectural decisions.
- **Note:** AI tools are strictly prohibited during lab tests and exams. **We strongly recommend regular coding practice to ensure individual proficiency for these assessments.**

1. Project's Background

In programming courses such as CS102, each lab test or exam typically requires students to submit multiple source files (e.g., Q1a.java, Q1b.java). With large class sizes, manual grading becomes extremely time-consuming, error-prone, and inconsistent. In addition, instructors often need to repeatedly compile, run, and inspect student submissions under strict grading rules.

An Auto Grading System aims to automate this workflow. The system ingests student submissions in bulk, validates file structure and naming conventions, compiles and executes code against predefined test cases, captures outputs and errors, and produces structured grading reports. This project emphasizes Object-Oriented Programming (OOP) design, file processing, process execution, and robustness—key skills for real-world software engineering.

2. Learning Objectives

By completing this project, students will:

- Design and implement a Java application applying core OOP principles (encapsulation, inheritance, polymorphism, abstraction).
- Work with file systems to process large volumes of student submissions automatically.
- Execute external processes (javac / java) programmatically and capture outputs and errors.
- Design modular grading pipelines with extensible components.
- Implement robust validation, logging, and reporting mechanisms.
- Build a usable console, GUI or web tool that reflects real teaching and grading scenarios.

3. Problems

An exam sample, code resources, student submissions (6 students' submissions), scoresheet (CSV file), and tester files are provided. The marking procedure involves several steps, each requiring manual checking and processing, which makes the overall process tedious and time-consuming.

Step 1 (Folder Arrangement): Before marking begins, instructors must verify the submitted files, as some students fail to follow the instructions. According to the exam guidelines, students should write their code in the provided resource folder named `RenameToYourUsername`, which contains subfolders (Q1, Q2, Q3) for the respective Java files. Each Java file (e.g., Q1a.java) must include the student's name and email at the top. Prior to submission, students are required to rename the folder to their username (e.g., `jiagu.wen.2025`), compress it into `jiagu.wen.2025.zip`, and then submit. The correct structure should be like:

`jiagu.wen.2025.zip`

```

└── jiagu.wen.2025
    ├── Q1
    │   ├── Q1a.java
    │   └── Q1b.java
    ├── Q2
    │   └── Q2.java
    └── Q3
        └── Q3.java
  
```

However, common issues occur, such as:

- **Forgotten folder renaming** e.g., jiagu.wen.2025.zip still contains RenameToYourUsername instead of jiagu.wen.2025.
- **Incorrect folder hierarchy**: the submitted zip file contains Q1, Q2, Q3 directly without a username folder. In worse cases, students also forget to include their name in the Java file header, leaving instructors unable to identify the author from the files alone. For example, student jing.lim.2021 submitted in this manner. Since multiple students may make similar mistakes, instructors must spend significant time manually verifying originality.

When such errors occur repeatedly across many students, instructors must manually inspect directory structures and source files to identify ownership and validity, significantly increasing grading time.

Step 2 (Tester files): Instructors must create tester files for each question and place them into the corresponding subfolders (Q1, Q2, ...). These tester files are compiled and run manually to generate scores. Importantly, instructors must design new test cases that differ from those in the original question files to prevent hard-coding. For instance, if a student hard-codes answers and the tester uses the same cases as the question file, the student could incorrectly receive full marks. To avoid this, instructors must prepare tester files with different test cases (e.g different inputs).

Step 3 (Score Calculation): Scores are computed by compiling and running the tester files for each question. Please check the given tester files contained in Tester-Files folder to see the marking rubric. For example, Q1aTester.java has 3 test cases. If the student can pass 1 case, then award 1 point. However, student programs may contain logical errors that lead to **infinite loops** or **excessively long execution times**. When such programs are executed:

- The grading process may block indefinitely.
- Instructors must manually interrupt execution, remove certain testing cases and force the tester to check the next test cases. If all the following test cases invoke infinite loops, instructors will need to manually calculate the score for this question based on the passing cases.

A single non-terminating program can significantly disrupt the entire grading workflow if not properly isolated.

Step 4: (Fill the ScoreSheet): After compilation and execution, instructors must manually record individual question scores, aggregate results across questions, and transfer final marks into a centralized score sheet. This process is mechanical but highly susceptible to human error, especially for large cohorts.

4. Requirements

Your task is to design and implement an **Auto Grading System** that addresses the challenges described in [Section 3](#).

4.1 Core Functional Requirements

The system must be able to:

- Enable batch grading of student submissions instead of requiring users to click and grade each submission individually.
- Validate submission structure, file naming conventions, and presence of required Java source files.
- Detect and report structural anomalies (e.g. missing a file/ missing a Qx folder) without terminating the grading process.
- Automatically correct common submission errors, such as incorrect folder names, improper folder hierarchy, and missing student names in Java file headers, where applicable.
- Safely terminate programs that exceed execution time limits.
- Automatically compute per-question and per-student scores.
- Export grading results in the provided CSV format.

4.2 Design and Code Quality Requirements

- The system must be implemented in Java.
- The design must demonstrate Object-Oriented Programming principles.

- Major responsibilities such as validation, compilation, execution, grading, reporting should be encapsulated in separate classes or modules.
- The system must be robust – failure of a single submission must not crash the entire application.

4.3 User Interface Requirements

- The application may be implemented as a console-based, GUI-based, or web-based system.
- Usability is more important than visual aesthetics.
- The interface must clearly communicate grading progress, detected errors, and final results to instructors.

5. Bonus Features

Bonus features are intended to encourage deeper thinking about grading fairness, robustness, and instructor experience. Bonus marks will be awarded based on **usefulness, correctness, and design quality**.

Examples of bonus features include, but are **not limited to**:

- Detection and handling of additional edge cases not explicitly listed in Section 3. **NOTE:** You are encouraged to create additional dummy submissions for testing, as only 6 sample student submissions are provided.
- Intelligent warnings for abnormal submissions (e.g., students submitting the original resource folder instead of their working folder).
- Automatic or parameterized generation of tester files to reduce hard-coding risks.
- Basic plagiarism detection.
- Support for grading multiple assessments using the same framework.
- Generate a detailed report summarizing detected anomalies, common mistakes, and other relevant issues. You are encouraged to identify and report additional anomaly types beyond those specified in Section 4.1. **NOTE:** you are free to decide the report format, no need to match the provided scorereport.csv.

Project Rubrics

	Evaluation Scale (10 – 30)			
	10 – 14	15 - 19	20 - 24	25 - 30
Code	There was no attempt to modularize the code with multiple classes and methods.	There was some attempt to modularize the code. A lot of repetitive code.	Most of the code were modularized with the exception of a few 'god' classes.	Applied good object modularization, methods perform one function only.
	There are multiple coding styles within the program. It does not adheres to the Java coding convention. There was no use of packages.	The coding style was consistent, but it does not adheres to the Java coding convention. There is Java packages but the intent of each package wasn't clear.	Most of the code follows Java coding convention with a few exceptions. Clear use of packages.	The code follows Java coding convention consistently. Clear use of packages.
	All configuration parameters (e.g. database URL) are hard-coded. Magic numbers were used.	Some configuration parameters (e.g. database URL) are hard-coded	All configuration parameters are externalized. There are occasional uses of magic numbers.	All configuration parameters are externalized. Constants are used instead of littering values all over.
	No obvious attempt to design the solution. It's a brute-forced solution.	Some attempt to do design. There are entity classes found.	Good attempt on doing object design. There was also the idea of separating the model from the view.	The design was clean. reader can easily figure out the intent of each class and the relationships between classes. It's a piece of art!

Application	The application is straightforward and is handled with an if-else statement. On top of that, it's buggy.	The application is straightforward and is handled with an if-else statement, or the application has some complex situation but has kinks all over.	The application has some complex scenarios and are reasonably handled. The app is mostly intuitive with some kinks.	Application has a lot of complex scenarios, and they are well-handled (e.g. lots of conditions are considered and handled) with versatile functions.
	The solution does not work or has lots of bugs (e.g. stack trace, crashes).	The solution somewhat works but has usability issues or occasional bugs.	The solution works with some usability or occasional bugs.	The app is intuitive (e.g. all prompts, alerts, statuses, etc. are well placed). The solution works perfectly.
	The application with/without UI is puzzling and/or hard to navigate.	The application with/without UI is decent with some usability issues or occasional bugs.	The application with/without UI is intuitive and user-friendly, with minimal usability issues	The application with/without UI is exceptional, polished, and highly user-centric.
Presentation	The presentation missed the points required.	The presentation covers 50% of the main points but has not elaborate well.	The presentation covers most or all the main points and has not elaborated all the points fully.	The presentation covers all the main points and elaborates well.
	The presentation deck was unclear, messy and with bad contrast or small fonts.	The presentation deck mostly decent with occasional flaws (e.g. font, contrast).	The presentation deck is decent. The team used it effectively to explain their project.	The presentation deck was well-put together. Diagrams are clear and illustrate the design.
	The presentation is not well-paced, and is hard to follow.	The presentation has some pacing issues and/or is frequently hard to follow.	The presentation was mostly well-paced. However, the team needs to switch laptops or restart apps.	The presentation was well-paced.
	The team has not prepared or rehearsed on the demo.	The demo flow was mostly too fast or confusing. There was insufficient time allocated for the demo.	The demo flow was decent with an occasional hiccups.	The demo showed the features well.
Q&A	Q&A: The team struggles to provide accurate answers, and responses lack consistency. For example: 1. Many questions are not answered well or accurately. 2. Team members often provide differing answers to the same question, causing confusion. 3. Responses are frequently unclear, incomplete, or require significant follow-up.	Q&A: The team can answer most questions adequately, though some responses lack depth or clarity. For example: 1. Most questions are answered correctly, but explanations may be basic or leave some room for improvement. 2. Team members are mostly consistent, though a few answers may vary slightly. 3. There may be a few complex questions that the team cannot fully address.	Q&A: The team answers nearly all questions accurately, demonstrating strong knowledge and consistency. For example: 1. Responses are thorough, well-explained, and require minimal follow-up. 2. Team members provide consistent answers to similar questions, showing alignment in understanding. 3. Only occasional, less-common questions might require slight clarification.	Q&A: The team provides exceptional responses, consistently delivering clear, comprehensive, and prompt answers. For example: 1. Answers are highly accurate, detailed, and address all aspects of each question with professionalism. 2. Team members are fully aligned, delivering consistent and coherent responses across all inquiries. 3. The team proactively addresses potential follow-up questions, ensuring a smooth, efficient Q&A experience.